

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

IoT Device Dashboard Backend API

Detailed project brief generated from the project record.

CANDIDATE

Rohan Desai

DURATION

3 days

TECHNOLOGY STACK

Java

HTML

CSS

React

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

Create a small Java REST API that serves simulated IoT device data and a minimal React page that consumes the API. The project evaluates the ability to understand requirements, design a simple backend, and integrate a frontend.

CANDIDATE Rohan Desai	TYPE backend	DURATION 3 days
---	------------------------------------	---------------------------------------

Primary Objectives

- Set up a Spring Boot project with a single controller.
- Create in memory data representing IoT devices.
- Build a React component that calls the API and renders a table.
- Write unit tests for the API endpoints.
- Document build/run steps.

Candidate Details

NAME

Rohan Desai

Technology Stack

Technologies, frameworks and tools expected for implementation.

Java

Technology / Tool

HTML

Technology / Tool

CSS

Technology / Tool

React

Technology / Tool

Functional Requirements

- GET /api/devices returns a list of device objects (id, name, status).
- GET /api/devices/{id} returns a single device or 404.
- POST /api/devices adds a new device with validation.
- React page displays the device list in a table.
- React page shows error message when API call fails.
- All data is stored in memory; persistence is not required.

Non-Functional Requirements

- API must respond within 200 ms for the in memory dataset.
- Input data must be validated (non empty name, allowed status values).
- Error responses must include a clear JSON error object.
- Code must follow Java naming conventions and React component best practices.
- Project must include a README with setup instructions.

Modules & Features

Major functional areas that make up the project.

API Server

Spring Boot application exposing device endpoints.

- Device controller
- In memory repository
- Validation layer
- Error handling

Data Model

Java classes representing device entities.

- Device POJO
- Enum for status
- DTOs for request/response

Sample Frontend

React page that consumes the API.

- Fetch hook
- Table component
- Error display
- Simple styling

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Setup

Initialize Git repo., Create Spring Boot starter project., Create React app with `create react app`., Add Maven and npm scripts to README.

- Initialize Git repo.
- Create Spring Boot starter project.
- Create React app with `create react app`.
- Add Maven and npm scripts to README.

2

Core API

Define Device POJO and Status enum., Implement in `memory repository`., Create DeviceController with GET endpoints., Add input validation for POST.

- Define Device POJO and Status enum.
- Implement in `memory repository`.
- Create DeviceController with GET endpoints.
- Add input validation for POST.

3

Frontend Integration

Create `fetch hook` to call `/api/devices`., Build DeviceTable component., Add error handling UI., Apply basic CSS for readability.

- Create `fetch hook` to call `/api/devices`.
- Build DeviceTable component.
- Add error handling UI.
- Apply basic CSS for readability.

4

Testing & Documentation

Write JUnit tests for controller., Run npm test (React lint)., Update README with usage examples., Create simple post mortem performance note.

- Write JUnit tests for controller.
- Run npm test (React lint).
- Update README with usage examples.
- Create simple post mortem performance note.

API Design

Key API endpoints and expected responsibilities.

GET

/api/devices

Return all devices

GET

/api/devices/{id}

Return device by id

POST

/api/devices

Create new device

Evaluation Criteria

Criteria that can be used to assess the completed project.

- Criterion: Correct API implementation | Description: Endpoints return expected JSON and status codes. | Weight: 25
- Criterion: React integration | Description: Frontend fetches data and renders table correctly. | Weight: 20
- Criterion: Code quality | Description: Follows language conventions and is well structured. | Weight: 15
- Criterion: Validation & error handling | Description: Invalid inputs are rejected with clear messages. | Weight: 15
- Criterion: Testing coverage | Description: Unit tests cover core API logic; basic frontend checks exist. | Weight: 15
- Criterion: Documentation | Description: README enables a reviewer to build, run, and test the project. | Weight: 10

Expected Deliverables

Artifacts and outcomes expected at project completion.

- Complete source code (Java + React)
- README with setup/run instructions
- API documentation (Markdown)
- JUnit test suite
- React lint report
- Performance note

Candidate Alignment

Project requirements compared with the candidate information available in the project record.

Rohan's background in embedded C++ and IoT gives him domain knowledge of devices. This project introduces Java for backend services and React for a simple UI, providing a realistic junior level challenge that matches the 3 day timeframe while evaluating problem solving, API design, and basic frontend integration.

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.